Primeira Lista de Exercícios

Tópicos Avançados de Redes Neurais Profundas e Aplicações
Prof. Gilson Antonio Giraldi

Lorran de Araújo Durães Soares

15 de julho de 2026

Introdução

Este documento apresenta a resolução da primeira lista de exercícios da disciplina *Tópicos Avançados em Redes Neurais Profundas e Aplicações*, ministrada pelo professor Gilson Antonio Giraldi. A lista é composta por três questões, que serão apresentadas ao longo deste documento, seguidas de suas respectivas resoluções.

O código desenvolvido para a realização dos exercícios está disponível no repositório `https://github.com/lorrantwo/LNCC`. A implementação foi desenvolvida em arquivos *Jupyter Notebook* (formato `.ipynb`), utilizando bibliotecas como NumPy [**?**] para manipulação de tensores, Matplotlib [**?**] para visualização dos resultados e PyTorch [**?**] para a definição, treinamento e avaliação dos modelos de inteligência artificial.

1 GANs para Parabolóide

Questão 1: Considere o parabolóide $P \subset \mathbb{R}^3$ representado por:

$$S : \mathbb{R}^2 \longrightarrow P; \quad S(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2 + 1) \tag{1}$$

Construa o conjunto de dados:

$$D = \{S(x_i^1, x_i^2) \in \mathbb{R}^3; \quad i = 1, \ldots, N\}$$

(a) Usando APENAS o conjunto de dados D, gere um modelo GAN que compute S;

(b) Tomando o gerador G, projete uma estratégia para avaliar a qualidade da representação através do erro quadrático médio (RMSE) e do erro absoluto médio (MAE). Discuta os resultados;

(c) Calcule ∇G através de diferenciação automática sobre um conjunto de pontos Z no espaço latente. Tomando a expressão (1), calcule ∇S analiticamente sobre Z e compare os resultados. Discuta seus achados.

Resolução:

Primeiramente, para a resolução desta questão, foi gerado o conjunto de dados D a partir do parabolóide definido na Equação 1. Utilizando a biblioteca NumPy [**?**], foram amostrados 5000 pontos do domínio $\mathcal{X} = [-2, 2] \times [-2, 2]$ a partir de uma distribuição uniforme. Cada ponto $(x_1, x_2) \in \mathcal{X}$ foi mapeado pela função $S(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2 + 1)$, gerando o conjunto de dados $D = \{S(x_i^1, x_i^2)\}_{i=1}^{5000} \subset \mathbb{R}^3$. A Figura 1 ilustra os pontos amostrados no plano cartesiano e a superfície do parabolóide correspondente.

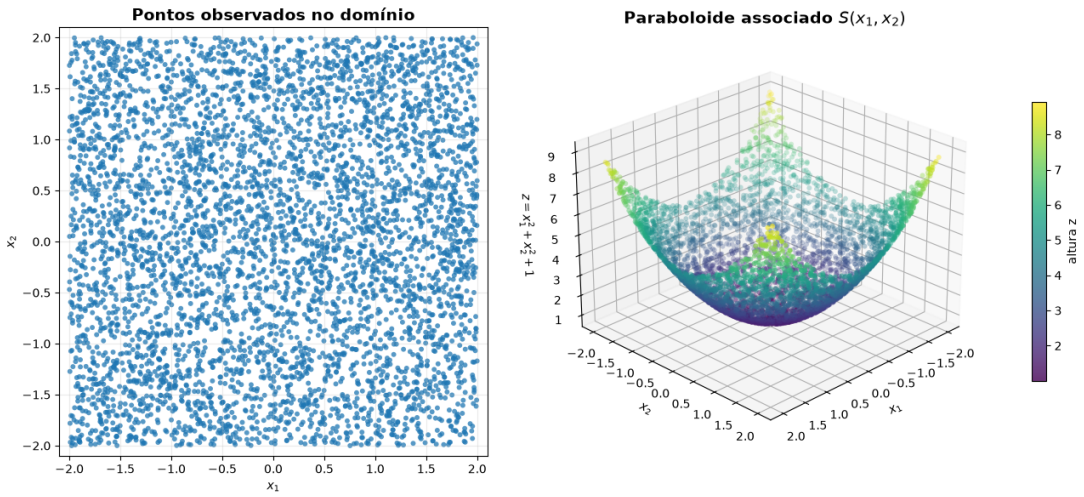


Figura 1: Pontos do domínio $\mathcal{X}$ amostrados uniformemente e o respectivo parabolóide gerado.

Desse conjunto, 80% das amostras foram destinadas ao treinamento da GAN, enquanto os 20% restantes foram utilizados como conjunto de teste. Como o conjunto de dados possui três coordenadas, sendo a terceira definida pela soma dos quadrados das duas primeiras acrescida de uma constante, seus valores tendem naturalmente a apresentar uma escala maior. Para minimizar esse desbalanceamento entre as variáveis de entrada, os dados foram normalizados utilizando a padronização Z-Score:

$$z = \frac{x - \mu}{\sigma}, \tag{2}$$

em que μ e σ representam, respectivamente, a média e o desvio padrão de cada coordenada, calculados apenas sobre o conjunto de treinamento.

Com os dados preparados, foi definida a arquitetura da GAN. A Figura 2 apresenta as arquiteturas da rede Geradora e da rede Discriminadora, definidas empiricamente.

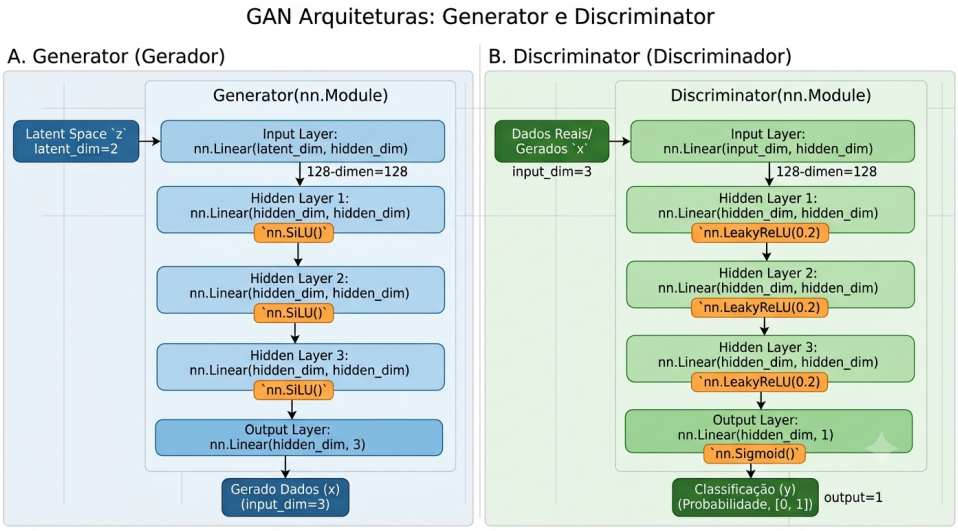


Figura 2: Arquiteturas da rede Geradora e da rede Discriminadora.

A rede Geradora possui cinco camadas totalmente conectadas: uma camada de entrada com dimensão 2, correspondente ao espaço latente, três camadas ocultas com 128 neurônios cada e uma camada de saída

com três neurônios, correspondentes às coordenadas (x_1, x_2, x_3) do parabolóide gerado.

A rede Discriminadora também possui três camadas ocultas com 128 neurônios cada. As camadas intermediárias utilizam a função de ativação LeakyReLU, definida na Equação 3:

$$\text{LeakyReLU}(x) = \max(\alpha x, x), \tag{3}$$

com $\alpha = 0.2$. A camada de saída possui um único neurônio, seguido pela função de ativação sigmoide, definida na Equação 4:

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \tag{4}$$

A saída do discriminador, denotada por $D(x) \in [0, 1]$, representa a probabilidade de uma amostra pertencer ao conjunto de dados reais. Valores próximos de 1 indicam maior confiança de que a amostra seja real, enquanto valores próximos de 0 indicam que ela foi gerada pela rede Geradora.

Para a otimização de ambas as redes foi utilizado o algoritmo Adam, definido pelas Equações 5–8, com taxa de aprendizado igual a 10^{-4} e parâmetros $\beta_1 = 0.9$ e $\beta_2 = 0.999$, definidos empiricamente.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \tag{5}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \tag{6}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \tag{7}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \tag{8}$$

em que g_t representa o gradiente da função de custo em relação aos parâmetros θ, η é a taxa de aprendizado e ε é uma constante utilizada para garantir estabilidade numérica.

A função de custo utilizada no treinamento da GAN foi baseada na *Binary Cross-Entropy* (BCE), uma função de erro empregada em problemas de classificação binária. Considerando um rótulo verdadeiro $y \in \{0, 1\}$ e a probabilidade estimada pela rede $\hat{y}$, a BCE é definida como:

$$\mathcal{L}_{BCE}(y, \hat{y}) = -\left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\right]. \tag{9}$$

No treinamento da GAN, o discriminador é responsável por classificar amostras reais como pertencentes à classe $y = 1$ e amostras geradas pelo gerador como pertencentes à classe $y = 0$. Dessa forma, a função de custo do discriminador, obtida a partir da BCE, é dada por:

$$\mathcal{L}_D = -\frac{1}{B} \sum_{i=1}^{B} \left[\log(D(x^{(i)})) + \log(1 - D(G(z^{(i)})))\right], \tag{10}$$

onde $x^{(i)}$ representa uma amostra real do conjunto de treinamento, $z^{(i)}$ representa um vetor amostrado do espaço latente e B é o tamanho do lote.

Por outro lado, durante a atualização da rede Geradora, o objetivo é produzir amostras que sejam classificadas pelo discriminador como reais. Assim, a função de custo do gerador é definida por:

$$\mathcal{L}_G = -\frac{1}{B}\sum_{i=1}^{B}\log(D(G(z^{(i)}))). \tag{11}$$

O treinamento foi realizado durante 300 épocas, utilizando $k = 1$, em que k representa o número de atualizações da rede Discriminadora realizadas para cada atualização da rede Geradora em cada lote (*batch*) de treinamento. Durante o treinamento, a rede Geradora recebe como entrada vetores latentes $z \in \mathbb{R}^d$, amostrados da distribuição uniforme $p_z = U(-2, 2)$. Esses vetores pertencem a um espaço latente independente do espaço dos dados e são utilizados como fonte de aleatoriedade para a geração de novas amostras. A partir desses vetores, a rede Geradora aprende uma transformação capaz de produzir amostras que seguem a distribuição dos dados reais. O procedimento completo de treinamento adotado é apresentado no Algoritmo 1.

Algorithm 1 Treinamento da GAN

Require: Conjunto de treinamento D, gerador G, discriminador D, número de épocas N, tamanho do lote B, número de atualizações do discriminador $k = 1$

for época $= 1$ até N **do**
  for cada lote $\{x^{(i)}\}_{i=1}^{B}$ **do**
    for $j = 1$ até k **do**
      Amostrar um lote de exemplos reais $\{x^{(i)}\}_{i=1}^{B}$
      Amostrar um lote de vetores latentes $\{z^{(i)}\}_{i=1}^{B} \sim p_z$
      Gerar amostras falsas $\hat{x}^{(i)} = G(z^{(i)})$
      Calcular

$$\mathcal{L}_D = -\frac{1}{B}\sum_{i=1}^{B}\left[\log D(x^{(i)}) + \log\left(1 - D(\hat{x}^{(i)})\right)\right]$$

      Atualizar os parâmetros da rede Discriminadora utilizando Adam
    end for
    Amostrar um novo lote de vetores latentes $\{z^{(i)}\}_{i=1}^{B} \sim p_z$
    Gerar amostras falsas $\hat{x}^{(i)} = G(z^{(i)})$
    Calcular

$$\mathcal{L}_G = -\frac{1}{B}\sum_{i=1}^{B}\log D(\hat{x}^{(i)})$$

    Atualizar os parâmetros da rede Geradora utilizando Adam
  end for
end for

Após o treinamento, a Figura 3 apresenta a evolução da função de custo das redes Geradora e Discriminadora ao longo das épocas. Observa-se que, nas primeiras iterações, a perda da rede Geradora aumenta, indicando que a rede Discriminadora consegue distinguir com facilidade as amostras reais das amostras geradas. À medida que o treinamento evolui, a rede Geradora passa a produzir amostras mais realistas, reduzindo gradualmente sua perda, enquanto a perda da rede Discriminadora tende a estabilizar. Esse comportamento é consistente com o processo esperado de treinamento de uma GAN.

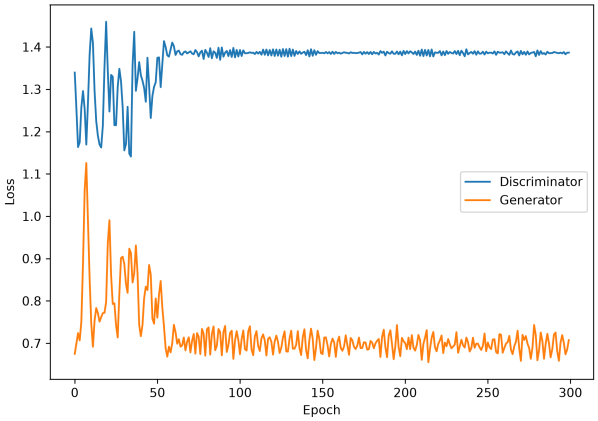


Figura 3: Evolução das funções de perda da rede geradora e da rede discriminadora.

Logo após, o modelo treinado foi utilizado para gerar novas amostras a partir do conjunto de teste. A superfície produzida pela rede Geradora é apresentada na Figura 4. Visualmente, observa-se que a GAN foi capaz de aprender a geometria do parabolóide, produzindo uma superfície bastante semelhante àquela presente no conjunto de dados original.

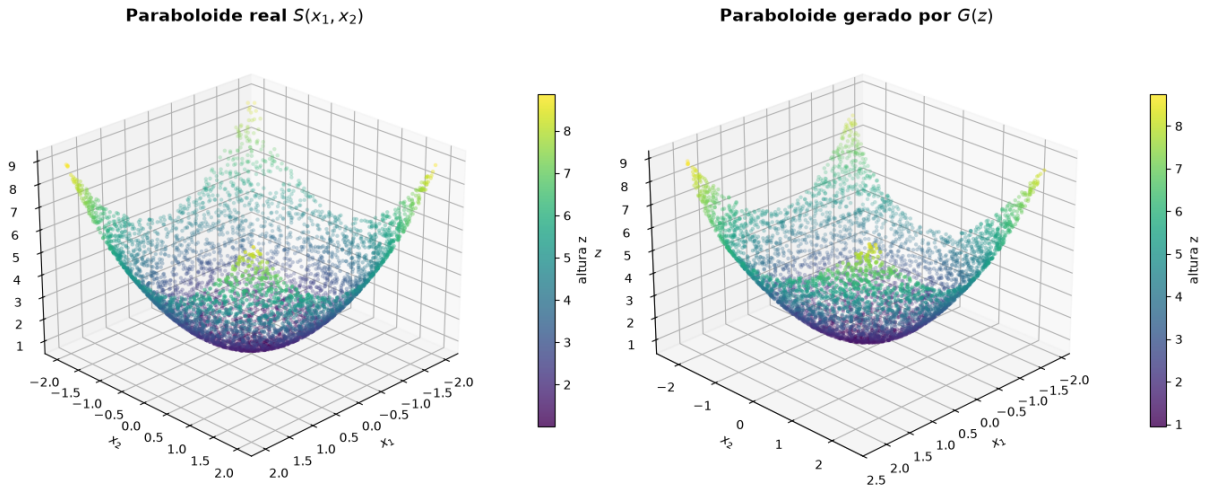


Figura 4: Comparação entre paraboloide do dataset e o gerado pela rede.

Com o intuito de analisar os resultados obtidos, novos vetores do espaço latente foram amostrados e fornecidos à rede Geradora para a criação de novas amostras. Como a GAN aprende a distribuição dos dados e não uma correspondência direta entre entrada e saída, a avaliação não foi realizada comparando pontos individuais gerados com pontos reais, mas sim verificando o erro geométrico das amostras geradas em relação à superfície esperada do parabolóide.

Para isso, foi calculado o erro da superfície gerada utilizando as métricas *Mean Absolute Error* (MAE), definida por:

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N} |e_i| \,, \tag{12}$$

e *Root Mean Squared Error* (RMSE), definida como:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2}, \tag{13}$$

onde e_i representa o erro geométrico entre o ponto gerado e a superfície ideal do parabolóide, dado por:

$$e_i = z_i - (x_i^2 + y_i^2 + 1). \tag{14}$$

Além disso, foram calculadas as versões relativas dessas métricas, normalizando os erros pelo valor médio da superfície gerada. Observa-se que a rede Geradora apresentou um erro geométrico absoluto de aproximadamente 0.35 para a métrica MAE e 0.44 para a métrica RMSE. Quando normalizados, esses valores correspondem a erros relativos de aproximadamente 14.05% e 16.87%, respectivamente. Esses resultados indicam que a GAN foi capaz de aprender a estrutura geral do parabolóide, gerando amostras próximas à superfície real, embora ainda apresente um desvio geométrico devido à dificuldade inerente do treinamento adversarial.

Por fim, considere a parametrização analítica do parabolóide definida pela função $S : \mathbb{R}^2 \to \mathbb{R}^3$,

$$S(x, y) = \left(x, y, x^2 + y^2 + 1\right). \tag{15}$$

Como essa função é diferenciável, seu jacobiano pode ser obtido analiticamente por

$$\nabla S(x, y) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2x & 2y \end{bmatrix}. \tag{16}$$

Da mesma forma, a rede Geradora define uma transformação $G : \mathcal{Z} \to \mathbb{R}^3$, que recebe vetores do espaço latente $\mathcal{Z}$ e produz pontos no espaço tridimensional. Seu jacobiano pode ser obtido por diferenciação automática, sendo dado por

$$\nabla G(z) = \frac{\partial G(z)}{\partial z}. \tag{17}$$

A comparação entre ∇S e ∇G permite investigar se o gerador aprendeu uma parametrização semelhante à função analítica do parabolóide. Caso isso ocorresse, seria esperado que as duas primeiras coordenadas produzidas pelo gerador reproduzissem aproximadamente as coordenadas do vetor latente, enquanto a terceira coordenada representasse a função $x^2 + y^2 + 1$. Entretanto, o treinamento de uma GAN não impõe essa restrição. O objetivo do gerador é apenas transformar vetores do espaço latente em amostras cuja distribuição seja semelhante à distribuição dos dados reais, não existindo qualquer obrigação de preservar as coordenadas do espaço latente durante essa transformação.

A Tabela 1 apresenta a comparação entre os jacobianos da função analítica e da rede Geradora. Para quantificar a diferença entre os jacobianos, foi utilizada a norma de Frobenius da matriz diferença entre ∇G e ∇S. Para cada ponto avaliado, essa métrica é definida por

$$\|\nabla G - \nabla S\|_F = \sqrt{\sum_{i=1}^{3}\sum_{j=1}^{2}\left(\frac{\partial G_i}{\partial z_j} - \frac{\partial S_i}{\partial x_j}\right)^2}, \tag{18}$$

em que a norma de Frobenius corresponde à raiz quadrada da soma dos quadrados de todos os elementos da matriz diferença entre os jacobianos. Quanto menor esse valor, maior é a semelhança entre as transformações locais realizadas pela função analítica e pela rede Geradora. Como essa métrica é calculada para cada amostra avaliada, foi considerada a média dessas normas ao longo de todos os pontos analisados, obtendo-se uma medida global da discrepância entre os dois jacobianos.

Observa-se que a norma média de Frobenius da diferença entre os jacobianos foi igual a 2,12, indicando diferenças significativas entre as duas transformações. Em particular, nota-se que as derivadas das duas primeiras coordenadas diferem substancialmente da parametrização analítica. Por exemplo, enquanto para a função S tem-se $\partial x/\partial a = 1$ e $\partial y/\partial b = 1$, a rede Geradora apresentou valores médios próximos de $0,00$ e $-0,03$, respectivamente. De maneira semelhante, as derivadas cruzadas, que são nulas na parametrização analítica, assumiram valores elevados no gerador, como $\partial x/\partial b = -0,93$ e $\partial y/\partial a = -1,14$. Por outro lado, as derivadas associadas à terceira coordenada apresentaram diferenças relativamente menores, com valores médios de $-0,03$ e $-0,24$ para o gerador, enquanto a função analítica apresentou $-0,05$ e $-0,06$.

Tabela 1: Comparação entre os jacobianos da função analítica S e da rede Geradora G.

	$\partial x/\partial a$	$\partial x/\partial b$	$\partial y/\partial a$	$\partial y/\partial b$	$\partial z/\partial a$	$\partial z/\partial b$
$G(a,b)$	0.0010	-0.9281	-1.1413	-0.0334	-0.0333	-0.2426
$S(a,b)$	1.0000	0.0000	0.0000	1.0000	-0.0546	-0.0631
$G - S$	-0.9990	-0.9281	-1.1413	-1.0334	0.0213	-0.1795

Métrica	Valor
Norma média de Frobenius	2.1233
Desvio padrão	0.1006

Esse comportamento pode ser visualizado na Figura **??**, que ilustra a correspondência entre os vetores do espaço latente e as duas primeiras coordenadas produzidas pelo gerador. Observa-se que os pontos não permanecem nas mesmas posições do espaço latente, evidenciando que a rede não aprendeu a identidade nas duas primeiras coordenadas. Em vez disso, o gerador aprendeu uma transformação não linear do espaço latente, utilizando os vetores apenas como uma fonte de aleatoriedade para produzir amostras pertencentes à distribuição do parabolóide. Esse resultado está de acordo com o objetivo do treinamento adversarial de uma GAN, que busca reproduzir a distribuição dos dados e não uma parametrização específica da superfície.

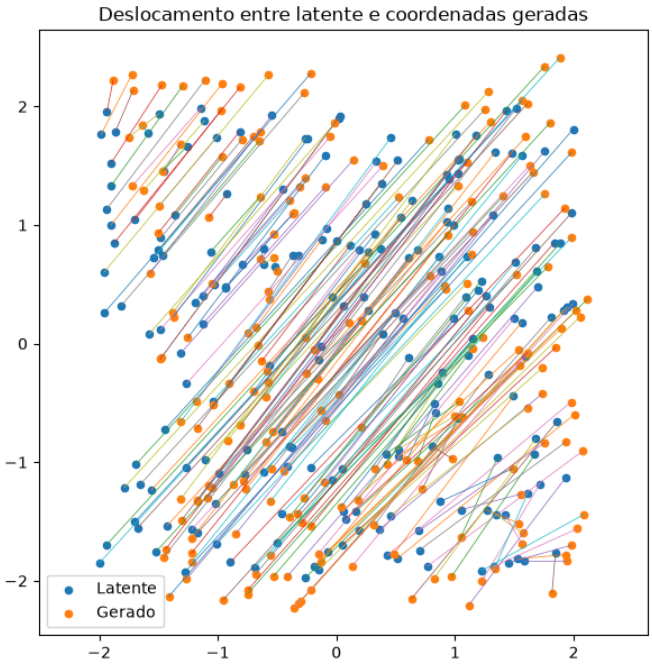


Figura 5: Correspondência entre os vetores do espaço latente e as duas primeiras coordenadas produzidas pela rede Geradora. As setas indicam o deslocamento de cada vetor latente para as coordenadas (x, y) geradas, evidenciando que a rede aprende uma transformação do espaço latente em vez de preservar sua parametrização.

Além desses fatores, deve-se considerar que reproduzir a geometria diferencial da superfície é uma tarefa mais complexa do que apenas reproduzir sua distribuição de pontos. A função objetivo de uma GAN clássica não impõe restrições para que o gerador preserve a estrutura local do *manifold*, como seus gradientes ou espaços tangentes. Dessa forma, mesmo quando as amostras geradas pertencem aproximadamente ao parabolóide, pequenas oscilações locais podem fazer com que os gradientes diferenciem significativamente daqueles da superfície analítica [**?**].

2 GANs com Operações Convolucionais

Questão 2: Escolha uma base de dados de imagens e treine um modelo GAN sobre ela, com um gerador utilizando operações convolucionais.

(a) Considere dois critérios de parada diferentes e compare-os;

(b) Escolha duas métricas de qualidade de imagem e compare-as para analisar o resultado da GAN.

Resolução:

Para a realização deste experimento, foi utilizado o conjunto de dados *MNIST* (*Modified National Institute of Standards and Technology*) [], composto por imagens em escala de cinza de dígitos manuscritos. Cada imagem possui dimensão de 28×28 pixels e pertence a uma das dez classes correspondentes aos dígitos de 0 a 9. O conjunto de dados contém 70.000 imagens, sendo 60.000 destinadas ao treinamento e 10.000 ao teste. Neste trabalho, apenas o conjunto de treinamento foi utilizado para o treinamento da Rede Generativa Adversarial (GAN). A Figura **??** apresenta alguns exemplos de imagens presentes na base de dados.

Como a camada de saída da rede geradora utiliza a função de ativação *Tanh*, cujos valores pertencem ao

intervalo $[-1, 1]$, as imagens do MNIST, originalmente representadas no intervalo $[0, 1]$, foram normalizadas para o intervalo $[-1, 1]$ por meio da seguinte transformação:

$$x_{\text{norm}} = 2x - 1. \tag{19}$$

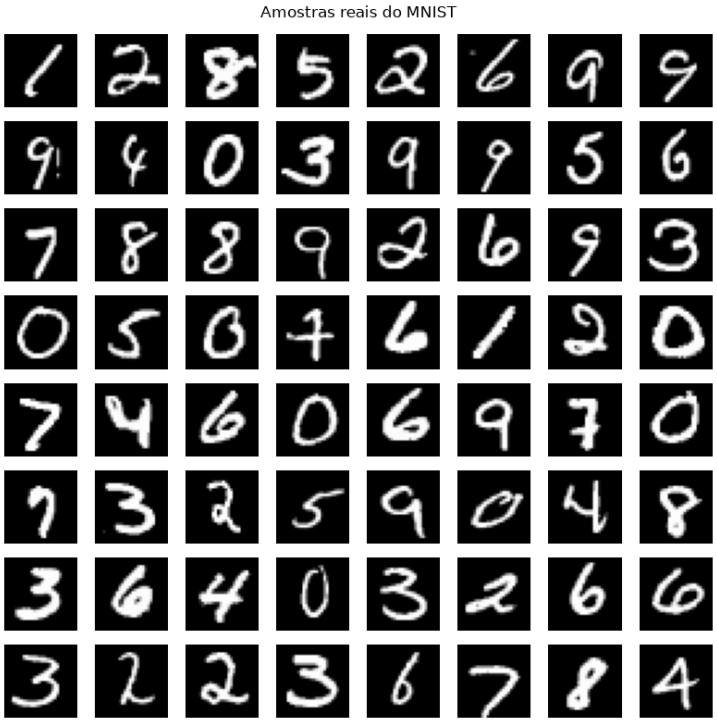


Figura 6: Exemplos de imagens do conjunto de treinamento do MNIST.

Considerando que o problema envolve a geração de imagens, foi adotada uma arquitetura convolucional para a GAN, inspirada na estrutura proposta pelas DCGANs (*Deep Convolutional Generative Adversarial Networks*). A arquitetura utilizada é ilustrada na Figura **??**.

A rede geradora recebe como entrada um vetor de ruído aleatório de dimensão 100, representado no formato $(100, 1, 1)$. A partir desse vetor, sucessivas camadas de convolução transposta (*ConvTranspose2d*) realizam o aumento gradual da resolução espacial até produzir uma imagem sintética de dimensão 28×28. Entre as camadas convolucionais são empregadas camadas de *Batch Normalization* e a função de ativação ReLU, enquanto a camada de saída utiliza a função *Tanh*, garantindo que os valores gerados permaneçam no mesmo intervalo dos dados normalizados.

A rede discriminadora recebe como entrada imagens reais ou sintéticas de dimensão 28×28 e realiza sucessivas operações de convolução para reduzir sua dimensão espacial e extrair características discriminativas. Nas camadas intermediárias é utilizada a função de ativação *LeakyReLU* com coeficiente negativo igual a 0,2, além de uma camada de *Batch Normalization*. A camada final produz um único valor escalar, indicando o grau de confiança da rede de que a imagem fornecida pertence ao conjunto de dados reais.

A Tabela **??** resume a arquitetura completa das redes geradora e discriminadora implementadas neste trabalho. Os hiperparâmetros da arquitetura foram definidos empiricamente, a partir de experimentos envolvendo diferentes configurações de número de filtros, tamanhos de kernel e demais parâmetros das camadas convolucionais.

Tabela 2: Arquitetura da GAN utilizada nos experimentos.

Rede	Camada	Filtros	Kernel	Stride	Padding
Geradora	ConvTranspose2d	256	7×7	1	0
	ConvTranspose2d	128	4×4	2	1
	ConvTranspose2d	1	4×4	2	1
Discriminadora	Conv2d	64	4×4	2	1
	Conv2d	128	4×4	2	1
	Conv2d	1	7×7	1	0

Na rede geradora, as duas primeiras camadas convolucionais transpostas são seguidas por operações de *Batch Normalization* e pela função de ativação ReLU. Já na camada de saída é utilizada a função *Tanh*. Na rede discriminadora, a primeira camada convolucional é seguida por uma função *LeakyReLU*; a segunda camada utiliza *Batch Normalization* e *LeakyReLU*; enquanto a camada final não emprega função de ativação, produzindo diretamente o escore utilizado pela função de perda durante o treinamento.

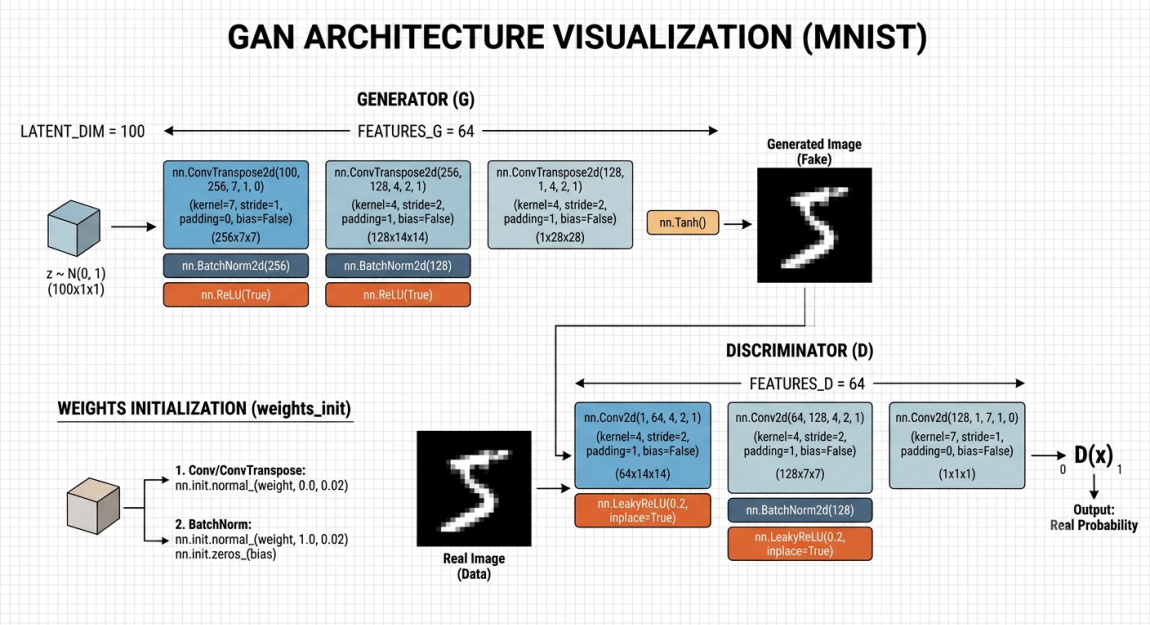


Figura 7: Arquitetura da GAN convolucional utilizada neste trabalho.

O treinamento foi então realizado de maneira análoga ao descrito na questão anterior (veja 1), utilizando o otimizador Adam, com mesmos hiperparâmetros, e as mesmas funções de custo a serem minimizadas.

Como estratégia de critérios de paradas adotados, utilizamos a estratégia de Early Stopping com patience igual a 5, ou seja, se não houvesse melhora do melhor resultado em 5 épocas o treinamento era encerrado. Como avaliar o resultado, foi empregada duas estratégias diferentes através de duas métricas diferentes que caracterizam os dois critérios de parada utilizadas. O primeiro treinava o modelo por 30 épocas sem parar, e o outro utilizava a métrica

3 Questão 3 - CNN para Estimativa de Idade

Utilize a base de dados UTKFace (`https://github.com/aicip/UTKFace`) ou outro repositório similar, e considere o problema de regressão de estimar a idade dada uma imagem facial usando um modelo CNN.

Aplique teoria de invariância (ou equivariância) para melhorar os resultados.

Referências

- Charles R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.
- John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9 (3):90–95, 2007.
- Qunwei Li, Bhavya Kailkhura, Rushil Anirudh, Jize Zhang, Yi Zhou, Yingbin Liang, T. Yong Jin Han, and Pramod Varshney. Mr-gan: Manifold regularized generative adversarial networks for scientific data. *SIAM Journal on Mathematics of Data Science*, 3(4):1197–1222, 2021.
- Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035, 2019.